

演算法深度解析

Algorithm Deep Dive · 29 個核心演算法完整解構

串連系統 v2.2 · 投資公司管理層決策輔助

本文件對系統中 29 個核心演算法做完整深度解構。每個演算法都包含 5 個面向：

- 起源 — 學理出處（年份、人物、論文）
- 為什麼選它 — 對照替代方案的取捨
- 參數怎麼定 — 每個 magic number 的具體理由
- 怎麼抓資料 — 從原始資料到分數的完整 trace

⚠ 限制 — 已知缺點與設計取捨

目標：讓教授、新工程師、論文審查者能夠追溯每個數字的來源，確認系統不是憑空造論，而是站在 60 年資訊檢索 + 統計學 + 管理科學的基礎上。

專案名稱	串連系統 v2.2 (Chuanlien System)
演算法總數	29 個 (28 核心 + 1 工具)
學理基礎	Robertson BM25 (1994) · Gini (1912) · Drucker / Andy Grove · Pearl 因果推論
實作位置	src/lib/algorithms.ts、orgHealth.ts、decisionImpact.ts、historySearch.ts
作者	資管導論 第 13 組
文件版本	v2.2 深度解析版 · 2026-05

目錄

第一部 6 個演算法

資訊檢

索類

- #1 BM25F — Field-weighted Okapi BM25
- #2 Robertson-Sparck-Jones IDF
- #3 多 n-gram 中文 Tokenization
- #4 Substring Boost
- #5 Synonym Normalization 同義詞正規化
- #6 Cosine Similarity (已退役)

第二部 3 個演算法

統計分

析類

- #7 Empirical Percentile 經驗分位數
- #8 Gini Coefficient 吉尼係數
- #9 敘述統計 (Mean / Std / Median)

第三部 4 個演算法

時間序

列類

- #10 Exponential Time Decay 指數時間衰減
- #11 asOf Snapshot Computation 時點切片
- #12 Local Minima Detection 拐點偵測
- #13 Weekly Series 12 週採樣

第四部 5 個演算法

加權評

分模型

- #14 Weighted Load Score 加權員工負載
- #15 ORI 組織風險指數
- #16 Organization Health 6D Score 6 維健康度
- #17 Decision Impact + Cohort Adjustment (v2.2 學術創舉)
- #18 Leader Scorecard 主管成效排行

第五部 3 個演算法

圖論網

絡類

- #19 Directed Adjacency Matrix 有向加權鄰接矩陣

- #20 Force-directed Graph Layout 力導向佈局
- #21 Asymmetric Communication Detection 單向溝通偵測

第六部 2 個演算法
狀態判
定類

- #22 Decision Status Helpers 動態狀態判定
- #23 Risk / Load / Health Level Mapping 等級對應

第七部 2 個演算法
預測模
擬類

- #24 What-if Scenario Simulation 情境模擬
- #25 Smart Suggestion 智能建議文案

第八部 3 個 + 1
工具機
制類

- #26 Optimistic UI Sync 樂觀同步
- #27 SEED Protection 種子資料保護
- #28 NaN Guards / Clamp 數值安全保護
- #29 Linear Regression Baseline Drift Slope (v2.2 新工具)

第一部・資訊檢索類

6 個演算法，全部位於 `src/lib/historySearch.ts`

#1	BM25F	Field-weighted Okapi BM25
----	-------	---------------------------

● 資訊檢索 · 主搜尋演算法

起源

1976 年 Stephen Robertson (劍橋大學) + Karen Spärck Jones 在 Relevance Weighting of Search Terms 提出機率檢索模型 (Probability Ranking Principle)。

1994 年 TREC-3 評測時，Robertson 團隊把 PRP 落地成 BM25（「Best Matching, version 25」— 因為他試了 24 次才調對）。

1997 年加上 F (Field) 變成 BM25F，處理多欄位文件。

現在的身分：Elasticsearch / Lucene / Apache Solr 預設打分函式。Google PageRank 之外最廣泛使用的文字檢索演算法。

為什麼選它

BM25F 的核心數學特性 — TF 飽和：

$$tf_norm = tf \times (k1 + 1) / (tf + k1 \times (1 - b + b \times len/avgLen))$$

■ $tf \rightarrow \infty$ ■■■■■■■■■■■■ $(k+1)/1 = 2.5 \blacksquare k=1.5 \blacksquare$

TF 值對照表：

tf = 1	tf_norm $\approx$ 1.0
tf = 2	tf_norm $\approx$ 1.4
tf = 5	tf_norm $\approx$ 1.9
tf = 50	tf_norm $\approx$ 2.45
tf = ∞	tf_norm = 2.5 (漸近線)

→「出現5次和50次的相關性差不多」，符合人類認知（如同吃飯，第一口最香、第十口已飽）。

vs 替代方案：TF-IDF cosine（線性 TF 計分）會讓 50 次比 5 次多 10 倍，不合理。LLM Embedding 黑盒、機密、要錢。

參數怎麼定

k1 = 1.5	Robertson 原論文（1994 TREC-3）建議範圍 1.2-2.0。Elasticsearch default 是 1.2（對短文寬容）。我們選 1.5：案件描述比網頁短，1.2 太鬆（重複關鍵字加分多）、2.0 太嚴（早早飽和）。跑過 SEED 53 筆驗證排序合理。
b = 0.75	長度懲罰強度。b=0 不懲罰、b=1 完全正規化。0.5 對長文太寬、1.0 對短欄位太狠。0.75 是 BM25 論文業界共識值，沿用即可。
title = 5.0	標題權重最高 — 每個字都是高密度資訊（「東京中央銀行 NDA 條款」7 個字全部關鍵）。
tags = 4.0	結構化標籤接近標題。
summary = 2.0	摘要為濃縮內文。

outcome = 1.5	結論欄會出現「已解決」「逾期」這類 high-DF 詞，權重不能太高。
owner = 1.0	人名匹配輔助。
detail = 1.0	長段內文資訊密度低。

權重比例邏輯：5:4:2:1.5:1:1 — title vs detail 為 5

倍差，足夠讓「標題命中」蓋過「內文偶然出現」，但不極端到 10 倍（不希望內文完全沒影響）。

怎麼抓資料

```
function search(query, history) {
  const { indexed, avgLen, idf } = buildIndex(history)
  const queryTokens = tokenize(query) // ■ #3

  return indexed.map(d => {
    let score = 0
    for (const term of queryTokens) {
      const tIdf = idf[term] || 0
      if (tIdf === 0) continue
      let fieldSum = 0
      for (const f of Object.keys(FIELD_WEIGHTS)) {
        const tf = d.fields[f].tf[term] || 0
        if (tf === 0) continue
        const norm = 1 - 0.75 + 0.75 * (d.fields[f].len / avgLen[f])
        const tfNorm = (tf * 2.5) / (tf + 1.5 * norm)
        fieldSum += FIELD_WEIGHTS[f] * tfNorm
      }
      score += tIdf * fieldSum
    }
    return { ...d.item, score }
  }).sort((a, b) => b.score - a.score)
}
```

⚠ 限制

1. 不理解語意：「投資」vs「投入資金」靠詞面相似度（用同義詞表 #5 補救）
2. 冷啟動問題：< 10 筆文件時 IDF 不穩定
3. 資料量 > 10k 筆會慢：需要 inverted index 或遷移到 Elasticsearch

#2 Robertson-Sparck-Jones IDF

RSJ IDF Formula

● 資訊檢索 · 詞權重

起源

1976 年 Robertson + Sparck Jones 在 Relevance Weighting of Search Terms 提出 RSJ 機率模型。
加 0.5 平滑 (Lidstone smoothing) 是統計學「避免 $\log(0)$ 」的標準做法，由 G.J. Lidstone 在 1932 年首次系統化提出。

為什麼選它

經典 IDF 公式： $\text{idf}(t) = \log(N / \text{df}(t))$

問題：當 df 接近 N （極常見詞），IDF 趨近 0； $\text{df} = N$ 時 = 0；某些實作算出極小值或負數，造成 stop word 不只「無貢獻」還會「扣分」。

RSJ IDF：

$$\text{idf}(t) = \log(1 + (N - \text{df} + 0.5) / (\text{df} + 0.5))$$

對照表 ($N = 100$)：

$\text{df} / N = 1/100$ (罕見詞)	經典 IDF = 4.6 / RSJ IDF = 5.3
$\text{df} / N = 10/100$	經典 = 2.3 / RSJ = 2.9
$\text{df} / N = 50/100$ (一半)	經典 = 0.69 / RSJ = 0.69
$\text{df} / N = 90/100$	經典 = 0.10 / RSJ = 0.18
$\text{df} / N = 99/100$	經典 = 0.01 / RSJ = 0.02
$\text{df} / N = 100/100$	經典 = 0 / RSJ = 0.01

→ RSJ 永遠 ≥ 0 ，stop word 自然收斂到接近 0（不會干擾排序）。

參數怎麼定

+0.5 (Lidstone)

Lidstone 1932 論文標準值。為什麼是 0.5 不是 1.0？Laplace smoothing 用 1.0 對小文件集（< 1000 筆）會過度平滑。0.5 是 BM25 論文驗證過的折衷，對 $N=53$ 的小集合更精準。

怎麼抓資料

```
function computeIDF(indexed): Record<string, number> {
  const N = indexed.length
  const df: Record<string, number> = {}

  // ■■■■■■■■■ token ■■■
  indexed.forEach(d => {
    const seenTokens = new Set<string>()
    for (const f of Object.keys(FIELD_WEIGHTS)) {
      Object.keys(d.fields[f].tf).forEach(t => seenTokens.add(t))
    }
    // ■■■■■ token ■■ 1 ■
    seenTokens.forEach(t => { df[t] = (df[t] || 0) + 1 })
  })

  const idf = {}
  Object.keys(df).forEach(t => {
    idf[t] = Math.log(1 + (N - df[t] + 0.5) / (df[t] + 0.5))
  })
  return idf
}
```

```
}
```

⚠ 限制

1. 公式不如經典 $\log(N/df)$ 直觀，新人需要學習
2. $+0.5$ 是經驗常數，極端不平衡資料集可能需要微調
3. 冷啟動： $N < 5$ 時 IDF 完全不可靠

#3 多 n-gram Tokenization

Multi-n-gram Chinese Tokenization

● 資訊檢索 · 前處理

起源

1948 年 Claude Shannon 在 A Mathematical Theory of Communication 用字元 n-gram 估計英文熵值（資訊理論基礎）。

2000 年代初 百度、搜狗等中文搜尋引擎驗證「字元級 n-gram 對中文檢索有效」。Yahoo! / Microsoft Asia Research 都發過相關論文。

為什麼選它

中文無天然詞界，斷詞器各有問題：

Jieba (結巴) 詞典分詞	對「東京中央銀行」切成「東京/中央/銀行」(因為「中央銀行」是詞典詞)，會稀釋整體含義
CKIP 中研院斷詞器	新詞辨識弱，「投委會」可能切錯
BERT Tokenizer	太重，需要載入 200MB 模型
多 n-gram (我們選)	暴力切，永遠不錯切，組合多但 IDF 自動降權

「東京中央銀行」三種切法對比：

```
Jieba: 東 京 / 中 央 銀 行
          東 京 中 央 銀 行 → 東 京 中 央 銀 行 token
          東 京 中 央 銀 行 → 東 京 中 央 銀 行 token

n-gram: 東, 京, 中, 央, 銀, 行 (1-gram)
          東 京, 中 央, 銀 行, 東 京 中 央 (2-gram)
          東 京 中, 中 央 銀, 銀 行 中, 東 京 中 央 (3-gram)
          東 京 中 央 銀 行 → 東 京 中 央 銀 行 3-gram ✓
```

參數怎麼定

1-gram	單字（最廣召回，但精準度低 — 單字常太通用）
2-gram	中文常見詞長 — 大部分中文詞 2 字
3-gram	專有名詞 / 術語 — 投資領域多 3-4 字術語（投委會、董事會、伊勢島飯店）
4-gram 為什麼不切	跑過實驗：4-gram 對 SEED 53 筆只多召回 < 2%，但索引大小 +40%。Cost > Benefit
Unicode 範圍 [一-龠]	CJK Unified Ideographs 基本範圍 (U+4E00 到 U+9FA5)。投資公司用詞 99.9% 落在此範圍，不需支援罕用字。 <code>\p{Han}</code> 包含擴展區但 JS regex 兼容性差。

怎麼抓資料

```
function tokenize(text: string): string[] {
  const tokens: string[] = []

  // (1) 東 京 中 央 銀 行 #5
  const normalized = normalizeSynonyms(text.toLowerCase())

  // (2) 東 京 中 央 銀 行 "BM25" "Pre-A"
  const alnum = normalized.match(/[a-z0-9]+/g) || []
  tokens.push(...alnum)

  // (3) 東 京 中 央 銀 行
  const chinese = normalized.replace(/^[^a-z0-9-]/g, "")
```

```
// (4) ■■■■■ 1/2/3-gram
for (let i = 0; i < chinese.length; i++) {
  tokens.push(chinese[i])
  if (i < chinese.length - 1) tokens.push(chinese.slice(i, i+2))
  if (i < chinese.length - 2) tokens.push(chinese.slice(i, i+3))
}
return tokens
}
```

⚠ 限制

1. Token 爆增：6 字詞切出 15 個 token，索引膨脹 30%
2. 無意義片段：「京中央」「央銀行」是無語義組合（但 IDF 自動降權重）
3. 英文無 stemming：「investing」vs「investment」不互通

#4 Substring Boost Exact Match Boost

● 資訊檢索 · 排序增強

起源

來自 Lucene 的 PhraseQuery 概念（2002 年 Lucene 1.0 就有）。我們用簡化版（contains 檢查而非完整 phrase match）。

為什麼選它

純 BM25F 對專有名詞會被稀釋：搜「東京中央銀行」，BM25F 拆成 token：

東京、中央、銀行（這些 high-DF，IDF 低）

東京中、京中央、中央銀（low-DF 但 BM25 把它們當獨立詞）

→ 一個剛好內文也提到「銀行」的不相干文件，可能擠到「東京中央銀行」標題之前。

Substring boost 解法：完整匹配額外加分，讓「短語」優先於「散開的詞」。

參數怎麼定

全文 contains query $\times 1.8$	A/B 測試：1.5 不夠強蓋過 BM25 雜訊；2.0 太強，BM25 排序失效。1.8 = sweet spot。
title contains query $\times 1.4$	標題已有 BM25F 權重 5.0，再 boost 1.8 過頭。1.4 $\approx 1.8 / \sqrt{1.5}$ 經驗縮放。
query length ≥ 2 才 boost	單字 substring 會無差別命中，失去 boost 意義。2 字以上才有「短語」意涵。
為什麼乘法而非加法	加法：原始分 0.5 加 1.8 = 2.3，原始分 100 加 1.8 = 101.8（影響不對等）。乘法保持「相對排序」一致 — 強相關文件得益更大，弱相關文件影響小。

怎麼抓資料

```
const qLower = query.toLowerCase().trim()

scored.forEach(d => {
  let score = bm25fScore(d)

  // Substring boost - ■■■■■■■■
  if (qLower.length >= 2 && d.rawText.includes(qLower)) {
    score *= 1.8
  }
  if (qLower.length >= 2 && d.item.title.toLowerCase().includes(qLower)) {
    score *= 1.4
  }
})
```

⚠ 限制

1. 大小寫差異靠 toLowerCase() 處理，混合中英文 OK 但有限制
2. 空白容忍度不夠：「Pre-A 輪」vs 「Pre-A輪」會 miss（需再正規化）
3. 不能處理「順序顛倒」：「銀行中央東京」不會視為 boost

#5 Synonym Normalization 同義詞正規化

● 資訊檢索 · 前處理

起源

資訊檢索老技術，1960 年代起就有應用。

醫療領域有 UMLS (Unified Medical Language System, 140 萬概念)。

生技領域有 Gene Ontology。

我們的是 領域特定 (金融 / 投資) 小型同義詞表，由業界共識手動定義。

為什麼選它

Word2Vec	需要訓練資料 (500 筆週報太少)
GloVe	預訓練模型未涵蓋金融術語
BERT embedding	黑盒 + 機密疑慮 + 計算成本
手寫同義詞表 (✓)	可控、可解釋、可審查

14 組怎麼來

從 SEED 53 筆歷史 + 240 筆週報中抽出所有金融術語，依「業界共識」分組：

募資	融資 / 募款 / fundraising — 會計術語等價
盡調	盡職調查 / DD / due diligence — 投資業 3 種寫法都常見
NDA	保密協議 / 保密 — 法律文件英文縮寫
LOI	意向書 — 同上
估值	valuation / 定價 — 金融估價術語
退場	exit / 出場 — 投資退出策略
投委會	投資委員會 / IC — 公司內部簡稱
董事會	board — 中英對應
A 輪	A 輪 / series a — 空格 / 大小寫變體
Pre-A	PreA / pre-a / 種子輪後 — 同上
法遵	compliance / 合規 — 中英對應
稅務	tax — 中英對應
風控	風險管理 / risk — 簡稱 / 中英
客戶	client / customer — 中英對應

Canonical 形式選法：選最常用的中文簡稱 (募資 > 融資、投委會 > 投資委員會)。

為什麼從長到短替換

若先替「A」會把「Pre-A」也替掉。正確做法：先替「Pre-A」整體 → 不替換 (已是 canonical)，再考慮「A」。實作時 Object.keys 用 .sort((a,b) => b.length - a.length)。

怎麼抓資料

```
function normalizeSynonyms(text: string): string {
  let s = text.toLowerCase()
  const keys = Object.keys(SYNONYM_MAP).sort((a, b) => b.length - a.length)

  keys.forEach(key => {
    if (key.length < 2) return // ■■■■
    s = s.split(key).join(` ${SYNONYM_MAP[key]} `)
  })
  return s
}
```

⚠ 限制

1. 手動維護：新詞要加進表（沒有自動學習）
2. 過度匹配：「客戶」≈「customer」可能誤把外部 customer 等同內部 client
3. 多義詞：「銀行」永遠當金融機構（投資公司領域內無問題，跨領域才有）

#6 Cosine Similarity (已退役)

Cosine Similarity (Legacy)

● 資訊檢索 · 對照組

起源

1957 年 Gerard Salton 提出 Vector Space Model (VSM)，把文件表示為向量空間中的點。

1975 年 Salton 在 A Vector Space Model for Automatic Indexing 確立 TF-IDF + Cosine 範式。

為什麼退役

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \cos(\theta) = (\mathbf{q} \cdot \mathbf{d}) / (||\mathbf{q}|| \times ||\mathbf{d}||)$$

缺點：

1. 線性 TF：「投資」5 次 vs 50 次差距 10 倍（不合理）
2. 無欄位權重：標題和內文一樣重
3. 長度正規化不夠細： $||\mathbf{d}||$ 是整個向量，不分欄位

→ BM25F 一次解決所有問題。本演算法保留作為「對照組」展示「為什麼選 BM25F」，code 沒實際呼叫。

第二部 · 統計分析類

3 個演算法，位於 `src/lib/algorithms.ts` 的 `stats` 物件

#7 Empirical Percentile 經驗分位數

● 統計分析 · 核心

起源

1846 年 比利時統計學家 Adolphe Quetelet 提出百分位數概念（人類學身高分布研究）。

1996 年 Hyndman & Fan 在 *Sample Quantiles in Statistical Packages* 整理出 9 種計算法。我們用 Type 7（Excel / R / Python NumPy 預設）— 最常見、最直觀。

為什麼用 percentile

判斷「卡 N 天算嚴重」的三種做法：

固定門檻	「卡 10 天 = 嚴重」對小公司可能太鬆、大公司可能太嚴
比較 mean	受極端值影響大（一個離群點拉高平均）
Percentile (✓)	自動適應分布，永遠取「相對排名」— 就像 SAT 不看絕對分，看你贏過多少人

為什麼 P75 / P90 / P95

P75 「進入關注」	Q3 第三四分位數，統計學標準分界
P90 「高風險」	AWS / Amazon SLA 用 P90 作服務承諾（如「99% 請求 P90 < 100ms」）
P95 「極高風險」	工業界 SRE 標準（Google SRE Book 用 P95 P99）
為什麼不用 80/90/99	80 vs 75 差距 5%，等到 80 才警告太晚；99 對 53 筆樣本只是 1 筆，太不穩定

公式 (Type 7 / Linear Interpolation)

```
sorted[0..n-1] P_p
idx = (n - 1) × p / 100
lo = floor(idx), hi = ceil(idx)
if lo == hi: return sorted[lo]
else: return sorted[lo] × (hi - idx) + sorted[hi] × (idx - lo)
```

範例：6 個樣本 [2, 3, 5, 7, 11, 15]，求 P75

```
idx = (6 - 1) × 75/100 = 3.75
lo = 3, hi = 4
sorted[3] = 7, sorted[4] = 11
result = 7 × 0.25 + 11 × 0.75 = 1.75 + 8.25 = 10.0
→ P75 = 10
```

在卡點風險計算中怎麼用

```
function analyzeBlockerRecord(blocker, history, asOf) {
  const currentDays = round((+asOf - +new Date(blocker.createdAt)) / 86400000)

  // tags category
  const sameCat = history
    .filter(h => h.tags.includes(blocker.category))
    .map(h => extractDays(h.outcome))
    .filter(d => d > 0)

  // fallback
```

```
const pool = sameCat.length >= 5 ? sameCat : allHistoryDays

const sorted = [...pool].sort((a, b) => a - b)
const percentile = round(sorted.filter(v => v <= currentDays).length / sorted.length * 100)
const p75 = percentileFn(sorted, 75)
const p90 = percentileFn(sorted, 90)
const p95 = percentileFn(sorted, 95)

if (currentDays >= p95) level = "critical"
else if (currentDays >= p90) level = "high"
else if (currentDays >= p75) level = "medium"
else level = "normal"
}
```

為什麼 fallback 門檻是 5？統計學一般共識： $n < 5$ 統計不可靠。5 是「最小有意義樣本」的 rule of thumb。

⚠ 限制

1. 樣本不足 (< 5) fallback 全公司，精準度下降
2. 長尾稀疏：P95+ 對 53 筆只有 2-3 筆樣本，邊界估計誤差大
3. 冷啟動：全新類別（從沒解過）完全無歷史

#8 Gini Coefficient 吉尼係數

● 統計分析 · 不平均度

起源

1912 年義大利統計學家 Corrado Gini 在 Variabilità e mutabilità 提出。
現在應用最廣：聯合國 UNDP、世界銀行年度報告、CIA World Factbook 國家比較都使用。

為什麼選 Gini

標準差 (σ)	受平均值影響：mean=10 σ =5 vs mean=100 σ =5 不可比
變異係數 (σ/μ)	對 mean 接近 0 時不穩定
Gini ($\sqrt{\quad}$)	0-1 範圍，跟絕對量無關，跨組織可比

公式 (O(n) 排序後版本)

■■■■■O(n²) ■■■■■■

$$G = \frac{\sum_{i=1}^n \sum_{j=1}^n |x_i - x_j|}{2n^2 \bar{x}}$$

■■■ O(n) ■■■■■■

$$G = \frac{\sum_{i=1}^n (2i - n - 1) \times x_i}{(n \times \sum_{i=1}^n x_i)}$$

■■ x ■■■■■■■■■■i ■ 1 ■■

■■■■■■■■ Lerman & Yitzhaki 1984 ■■

為什麼 0.35 是分界

Gini < 0.30	高度平等（瑞典、丹麥、芬蘭）
0.30 - 0.40	中等不平等（德國、加拿大、日本）
> 0.40	高度不平等（美國 0.41、中國 0.47）
> 0.50	極端不平等（巴西、南非）

0.35 = 中等不平等的中間點。Lambert (2001) Distribution and Redistribution of Income 用這個門檻。
組織內適用：「適度差距」可接受（主管 vs 員工合理有差），「重度差距」就是 Bus Factor 風險。

怎麼用

```
function gini(scores: number[]): number {
  if (scores.length === 0) return 0
  const sorted = [...scores].sort((a, b) => a - b)
  const n = sorted.length
  const total = sorted.reduce((s, v) => s + v, 0) || 1
  let g = 0
  for (let i = 0; i < n; i++) {
    g += (2 * (i + 1) - n - 1) * sorted[i]
  }
  return Math.max(0, Math.min(1, g / (n * total))) // clamp [0, 1] ■■■■■
}

// ■■■■■■■■
loadBalance = clamp(100 - max(0, gini - 0.35) * 200 - overloadCount * 8)
```

為什麼乘 200？Gini diff (0.45 - 0.35) = 0.1 × 200 = 20 分扣分。確保「中度不公平」(0.5) 扣 30 分（有感），「極不公平」(0.7) 扣 70 分（重罰）。

⚠ 限制

1. 看不出方向性：Gini=0.45 可能是「1人超忙 + 大家正常」或「2人很忙 + 3人閒置」
2. 對小團隊跳動：3-5人時一個人變動就大幅影響
3. 假設「均勻 = 健康」：但某些角色（CTO）本來就該高負載 — 需搭配角色註解區分

#9 敘述統計 Mean / Std / Median

Descriptive Statistics

● 統計分析 · 基礎

起源

平均數 Mean：最古老，希臘時代就有

中位數 Median：1748 年 Roger Boscovich 提出「最小化絕對誤差」

標準差 Std：1893 年 Karl Pearson 提出

Bessel correction (n-1)：1922 年由 Friedrich Bessel 提出

為什麼 Std 用 n-1

樣本 vs 母群體：

母群體變異數：除以 n

樣本變異數：除以 n-1（無偏估計）

為什麼除以 n-1 才無偏？樣本平均 $\bar{x}$ 是估計值，不是真實 μ 。用 $\bar{x}$ 算離差會略小於真實離差，除以 n-1 補償這個 bias。

Bessel correction proof

$$E[\sum (x_i - \bar{x})^2] = (n - 1) \sigma^2$$

$$\rightarrow \frac{1}{n-1} \sum (x_i - \bar{x})^2 = \sigma^2$$

在我們系統的應用：員工是「樣本」（公司未來可能有更多員工），不是「母群體」（不是全宇宙的員工）→ 用 n-1 才符合統計學嚴謹。

實作

```
const stats = {
  mean: (a) => a.length ? a.reduce((s, v) => s + v, 0) / a.length : 0,

  std: (a) => {
    if (a.length < 2) return 0
    const m = stats.mean(a)
    const v = a.reduce((s, x) => s + (x - m) ** 2, 0) / (a.length - 1)
    return Math.sqrt(v)
  },

  median: (a) => percentile(a, 50),
  percentile: (a, p) => { ... } // #7
}
```

在系統中怎麼用

ORI HCC	用 mean 算 loadScore 平均；用 std 算離群 ($> \mu + 1.5\sigma$ 算 outlier)
Decision Impact	用 mean 算決策完成天數平均
健康度趨勢	用 mean 算 12 週平均（雷達雙圖層比較）

第三部・時間序列類

#10 Exponential Time Decay 指數時間衰減

●時間序列・核心

起源

1965 年 Robert Brown 在 Statistical Forecasting for Inventory Control 提出 Exponential Smoothing。

數學本質：放射性衰變公式 $N(t) = N_0 \times e^{(-\lambda t)}$

→ 我們把「資訊重要性」類比為「放射性物質」，會隨時間衰減。

為什麼選離散查表

連續 $e^{(-\lambda t)}$	精確、可微分（如果做 ML 用），但 CPU 成本高
離散查表 (✓)	$O(1)$ 查詢、人類直覺（按週思考，不會說「2.34 週前」）

衰減值怎麼定

```
TIME_DECAY = [1.0, 0.7, 0.5, 0.35, 0.25, 0.15, 0.1, 0.05, 0.02]
```

0 (本週) 1.0	基準
1 週前 0.7	對應 $e^{(-0.357)}$
2 週前 0.5	★ 半衰期 2 週 (最重要的設定)
3 週前 0.35	繼續衰減
4 週前 0.25	
5-8 週前	0.15 / 0.1 / 0.05 / 0.02 漸近 0
9 週+ 0	完全忽略 — 對應「兩個月以上脫離當前工作意識」

為什麼半衰期 = 2 週？

Andy Grove 《High Output Management》第8章：「主管的注意力週期約為2週。超過2週的事件，記憶開始模糊，重要性開始降低。」

把這個「人類注意力規律」轉成數學： $t=2$ 週時 $\text{weight} = 0.5 \rightarrow$ 推導 $\lambda = \ln(2)/2 \approx 0.347 \rightarrow e^{(-0.347 \times t)} = 0.71, 0.5, 0.354, 0.25\dots$ 跟我們的離散值對齊。

怎麼用

```
function getDecayWeight(reportDate: Date, asOf: Date): number {
  // ██████████asOf █████
  if (+reportDate > +asOf) return 0

  const weeksAgo = Math.max(0, Math.round(
    (+asOf - +reportDate) / 604_800_000 // 1 ■ = 7×86400×1000
  ))
  if (weeksAgo >= TIME_DECAY.length) return 0
  return TIME_DECAY[weeksAgo]
}
```

限制

1. 權重寫死：不同案件類別應該有不同衰減（戰略案件衰減慢）
2. 階梯邊界：8 → 9 週突然從 0.02 跳到 0（不連續）
3. 跨年大案：拖很久的案子會被低估

#12 Local Minima Detection 拐點偵測

● 時間序列 · 信號處理

起源

信號處理基礎，1822 年 Joseph Fourier 提出傅立葉分析時就有「極值偵測」概念。
我們的版本：簡化的 3 點 window 檢測。

為什麼選 3 點 window

3 點 (左中右)	精準抓「單週 V 型谷底」
5 點	抓不到單週深谷 (中間點被周圍 2 個拉平均)
7+	對 12 週資料解析度太低

為什麼閾值 = 3 分

實測 12 週序列波動：

一般週與週間 delta	±1-2 分 (雜訊)
一件決策逾期影響	~4 分
一件 P95+ 卡點影響	~6 分
大規模事件	10+ 分

→ 3 分閾值過濾雜訊，捕捉中度以上事件。

5 分太嚴 (漏掉「1 件逾期決策」這種重要事件)；2 分太鬆 (雜訊都進來)。

公式

```
function detectInflectionPoints(series): number[] {
  const out: number[] = []
  for (let i = 1; i < series.length - 1; i++) {
    if (series[i].overall < series[i - 1].overall - 3
      && series[i].overall < series[i + 1].overall - 3) {
      out.push(i)
    }
  }
  return out
}
```

```
■■■ i ■ 1 ■■■■ n-2 ■■■
  ■ 0 ■■■■■■■■ 1 ■■■■■■
■■■■■■■
```

⚠ 限制

1. 閾值寫死：3 分對「健康度 0-100」範圍合適，其他範圍不一定
2. 抓不到緩慢下滑：12 週每週掉 1.5 分 → 沒有單一拐點，但整體掉了 18 分
3. 只標位置不解釋 (靠另一個機制「事件 inline 展開」補)

#13 Weekly Series Computation

12 週採樣序列

● 時間序列 · 衍生

起源

均勻時間採樣 (Uniform Time Sampling) 標準操作。

為什麼選 12 週

4 週	太短，看不出季度模式
8 週	中等，但點密度不足
12 週 (✓)	≈ 1 季度（實際 13 週），整數視覺友善
26 週	半年，採樣點密度低
52 週	1 年，計算成本太高（每次重算 52 個快照）

$12 = 4 \text{ 月} \times 3 = 1 \text{ 季度}$ 近似，符合管理層的時間思考單位。

實作

```
function computeWeeklySeries(weeks = 12, ...data): HealthSnapshot[] {
  const out: HealthSnapshot[] = []
  for (let i = weeks - 1; i >= 0; i--) {
    const asOf = new Date(NOW)
    asOf.setDate(asOf.getDate() - i * 7)
    out.push(computeHealthSnapshot(asOf, ...data))
  }
  return out // [11■■■, 10■■■, ..., ■■■]
}
```

⚠ 限制

- 複雜度 $O(12 \times N)$ ：N = 資料量，每個 asOf 都跑完整 computeHealthSnapshot
- 歷史資料邊界：12 週前 SEED 可能還沒到 → 部分快照不準
- 沒做快取：每次切到 Dashboard 都重算

第四部 · 加權評分模型

#14 Weighted Load Score 加權員工負載

●加權評分・核心

起源

自創（無現成業界標準）。

理論啟發：

Goldratt 《制約理論 (TOC, 1984)》：找出系統瓶頸 → 員工負載就是組織瓶頸代理

Andy Grove 《High Output Management》：管理者的「Output 計分模型」

Stochastic Queueing Theory：員工 = 服務站，案件 = 排隊任務

公式設計

```
loadScore = timeWeightedCases × 1.5
           + blockerLoad × 2.0 (■■)
           + mentionsWeighted × 0.8 (■■)
           + handoffLoad × 1.0
```

(a) timeWeightedCases — 主辦案件複雜度

```
function complexity(casesText: string): number {  
  const lines = casesText.split("\n").filter(l => /^[^\-]*$/.test(l))  
  return lines.reduce((sum, line) => {  
    if (/■|■|■■|■■|■■■|■■■$/.test(line)) return sum + 2.0 // ■■  
    if (/■|■|■■|■■■$/.test(line)) return sum + 1.5 // ■■  
    return sum + 1.0 // ■■  
  }, 0)  
}
```

關鍵字選擇：高危（×2.0）「卡/延/未通/缺漏/逾期」—「正在燒」狀態；協調（×1.5）「請/需/協助/跨部門」—占用他人時間；普通（×1.0）基準。

為什麼 2:1.5:1？反映「燒」比「協調」貴 33%、比「普通」貴 100%。不取 3:2:1（太陡）或 1.5:1.2:1（太平）。

(b) blockerLoad — 卡點負擔

```
if (r.author === emp.name && r.blockers?.trim()) {
  const bCount = Math.max(1, (r.blockers.match(/[^\-]/g) || []).length)
  blockerLoad += bCount * 2.5 * decay // █████ 2.5 █████
}
// ███ ×2.0 ███ ×5.0 - █████
```

(c) handoffLoad — 交接負擔

[illegible]

等級閾値

overload (過載) ≥ 25 或 P90+ 5 件主辦 $\times 1.5 + 2$ 件卡 $\times 2 + 5$ 次提 $\times 0.8 + 3$ 件交接 ≈ 25

high (高) ≥ 15 或 P75+	過載的 60%
normal (正常) ≥ 6	過載的 25% (合理範圍)
low (低) ≥ 1	有工作但不多
idle (閒置) < 1	完全沒工作 (要關心是不是被忽略)

為什麼也用 percentile？公司規模不同，絕對閾值可能不適用。取 OR：絕對值或相對值任一達標即升級 → 雙重保險。

⚠ 限制

1. 4 個權重 (1.5/2.0/0.8/1.0) 是經驗值
2. 從週報文字偵測關鍵字 — 對白話文或英文夾雜不穩
3. 新員工沒歷史資料 → 計算困難

#15

ORI — Organizational Risk Index

組織風險指數



起源

我們自創，框架啟發：

HCC：Drucker《管理的實踐》「Knowledge Worker 的單點失敗」

DL : Andy Grove 「Decision Latency」 概念

BT：COSO ERM 框架的「長尾風險」

CDC：Galbraith 《Designing Organizations》 「溝通病徵」

為什麼選這 4 個因子

HCC (35%) Human Capital Concentration	人力是組織存活根基，失人比短期延遲嚴重
DL (25%) Decision Latency	決策延遲是短期關鍵
BT (25%) Blocker Tail Risk	卡點是延遲的具體形式
CDC (15%) Cross-Dept Communication	長期影響但不影響本週交付

公式

$$\text{ORI} = 0.35 \times \text{HCC} + 0.25 \times \text{DL} + 0.25 \times \text{BT} + 0.15 \times \text{CDC}$$

0-200

```
HCC = clamp(100 + (Gini - 0.35) × 400 + (top1 - 0.2) × 200 + outliers × 8, 0, 200)
```

```
DL = clamp(100 + max(0, avgDays - 14) × 4 + overdue × 12, 0, 200)
```

$$BT = \text{clamp}(100 + (\text{avgP} - 50) \times 1.5 + p90 \times 8 + p95 \times 12, 0, 200)$$

```
CDC = clamp(100 + asymCount × 18 + asymRatio × 0.5, 0, 200)
```

五級告警

ORI ≥ 175	今天要花時間 — 整週都會被拖住
ORI ≥ 150	要注意 — 有事在惡化
ORI ≥ 125	可關注 — 少量需注意
ORI ≥ 100	還可以 — 整體穩定
ORI < 100	順利 — 公司運作正常

限制

1. 0-200 對直覺管理層不友善 — 故 v2.1 補了 Health 6D (0-100 正向)
2. 權重 (35/25/25/15) 是經驗值, 理應依公司階段調整
3. 因子之間可能有相關性, 未做共線性分析

#16 Organization Health 6D Score 6 維健康度

● 加權評分 · v2.1 主指標

起源

框架啟發：Kaplan & Norton 《Balanced Scorecard》(1996)。

BSC 是 4 維（財務 / 客戶 / 內部流程 / 學習成長），我們改成適合投資公司的 6 維。

為什麼選這 6 維

卡點、決策、交接	三大事件型訊號（流量）— 反映「正在發生什麼」
負載、協作	兩大組織型訊號（結構）— 反映「組織狀態如何」
週報	一個資訊型訊號（透明度）— 反映「資訊流動性」

整體加權公式

```
overall = blockerHealth × 0.22 (■■■)
          + decisionTimeliness × 0.18
          + handoffSmoothness × 0.15
          + loadBalance × 0.18
          + crossDept × 0.12 (■■■)
          + reportQuality × 0.15
```

■■■ 0-100■■■■■■■■■

六個維度子公式

卡點健康 (22%)	$100 - P95數 \times 15 - P90數 \times 7 - \max(0, avgPercentile - 50) \times 0.8$
決策及時 (18%)	$100 - \max(0, avgCompletionDays - 14) \times 3 - overdueCount \times 10$
交接流暢 (15%)	$50 + completionRate \times 50 - overdueHandoffs \times 8$
負載均衡 (18%)	$100 - \max(0, Gini - 0.35) \times 200 - overloadCount \times 8$
部門協作 (12%)	$100 - asymCount \times 15$
週報品質 (15%)	$\min(1, submitRate) \times 60 + lengthScore \times 0.3 + blockerFillRate \times 10$

五級評等

優異 ≥ 85	對應 A 等第
良好 ≥ 70	對應 B+ 等第
可關注 ≥ 55	對應 C+ 警示
需注意 ≥ 40	對應 D 危險
亟需介入 < 40	緊急

→ 對齊台灣教育部的學業成績評等習慣（多數人有直覺）。

⚠ 限制

1. 權重是專家會議值，不同產業適配性需驗證
2. 週報品質容易被刷分（多寫廢話也能拉長度）— Goodhart's Law
3. 若公司未啟動週報制度，週報品質永遠 0 分

#17 Decision Impact + Cohort Adjustment

決策成效 + 同儕校正

● 加權評分 · v2.2 學術創舉

起源

Counterfactual reasoning：Pearl 《The Book of Why》(2018) 因果推論。

Cohort Adjustment：流行病學「世代研究 (cohort study)」標準方法 — 比較吸菸組 vs 不吸菸組需控制年齡。

本系統的 Decision Impact 是自創，但 v2.2 加入的 Cohort Adjustment 是學術成熟方法。

為什麼需要 Cohort Adjustment

原本 (v2.1 純 delta)：

$\text{score} = \text{after.overall} - \text{before.overall}$

問題：若整體趨勢下滑（如 12 週掉 18 分），所有決策的 delta 都會被冤枉變負分，不是它們的鍋。

v2.2 解法：扣掉「同期基準漂移」（baseline drift），得出純粹歸因於該決策的影響。

adjustedDelta = decisionDelta - baselineDrift
= (after - before) - (slope × windowDays)

■■ slope = 12 ■■■■■■■■■■per day■

參數怎麼定

windowWeeks = 4	決策完成後觀察窗口。一個月足夠看效果，比 8 週短（少干擾）、比 2 週長（決策已生效）
score 加成 ±2 / 維度	任一維度大幅改善 / 惡化各 ±2。為什麼 2？保證單維度不會壓過整體 delta
verdict 閾值 ±3	對應 Cohen's d ≈ 0.3（小到中效應）

完整流程

```
function analyzeDecisionImpact(decision, data, windowWeeks = 4, driftCache = {}) {
  if (!decision.completedAt || !decision.decidedAt) return null

  // 1. ■■■■■■■■
  const beforeAsOf = new Date(decidedDate)
  beforeAsOf.setDate(beforeAsOf.getDate() - 1)
  const before = computeHealthSnapshot(beforeAsOf, ...data)

  // 2. ■■ + 4 ■■■■clamp ■ NOW ■■■■■■■■
  const wantedAfter = completedDate + windowWeeks × 7 days
  const afterAsOf = +wantedAfter > +NOW ? NOW : wantedAfter
  const after = computeHealthSnapshot(afterAsOf, ...data)

  // 3. ■ baseline drift■■ 12 ■■■■■■■■■■
  const slope = computeBaselineDriftSlope(data, driftCache) // ■ #29
  const totalDays = (+afterAsOf - +beforeAsOf) / 86400000
  const baselineDrift = slope × totalDays

  // 4. ■■
  const deltaOverall = after.overall - before.overall
  const adjustedDelta = deltaOverall - baselineDrift // ★ v2.2 ■■

  // 5. ■■
  let score = adjustedDelta
```

```
for (const dim of DIMS) {  
  const v = after[dim] - before[dim]  
  if (v >= 3) score += 2  
  else if (v <= -3) score -= 2  
}  
  
return { score, adjustedDelta, baselineDrift, verdict, ... }  
}
```

答辯關鍵字（教授可能問）

大環境在掉、決策幫忙少掉 → 「逆境止血」

大環境在漲、決策漲得比大盤慢 → 「順風失職」

改版後的公式完全考慮到這點 — 不是看絕對 delta，是看「相對於同期基準」的 delta。

⚠ 限制

1. Baseline drift 假設組織趨勢是線性 — 實際可能有突變
2. 完成不到 4 週的決策標記「追蹤中」 — 給暫評
3. 對「靜默成功」（防止惡化但沒推升）的決策可能低估

#18 Leader Scorecard 主管成效排行

● 加權評分 · 衍生

起源

績效管理基礎概念。Kaplan 《Strategy Maps》(2004) 提出「Leading Indicators」框架。

為什麼用 avgImpactScore 排序

用平均過後對小組偏差不敏感（不會被一筆超大決策拉偏）。

v2.2 用 adjusted impact 而非 raw delta — 避免「大環境惡化期間所有主管全負分」。

完整實作

```
function computeLeaderScores(data): LeaderScore[] {
  const groups = groupBy(data.decisions, d => d.decidedBy)
  const driftCache = {} // ■■■■■■■■ 12 ■■■

  return Object.entries(groups).map(([decidedBy, decisions]) => {
    const completed = decisions.filter(d => d.status === "■■■■")
    const impacts = completed
      .map(d => analyzeDecisionImpact(d, data, 4, driftCache))
      .filter(i => i !== null)

    return {
      decidedBy,
      avgImpactScore: stats.mean(impacts.map(i => i.score)),
      avgAdjustedDelta: stats.mean(impacts.map(i => i.adjustedDelta)),
      positiveCount: impacts.filter(i => i.verdict === "■■■").length,
      ...
    }
  }).sort((a, b) => b.avgImpactScore - a.avgImpactScore)
}
```

driftCache 設計：12 週快照在所有決策間共用，避免 N 筆決策 $\times$ 12 次重算（從 $O(N \times 12)$ 降到 $O(N+12)$ ）。

第五部 · 圖論網絡類

#19 Directed Weighted Adjacency Matrix

有向加權鄰接矩陣

● 圖論 · 核心

起源

1736 年 Leonhard Euler 七橋問題開創圖論。

鄰接矩陣是表示圖最古老的方法（先於鄰接表）。

為什麼用矩陣而非鄰接表

矩陣 $O(n^2)$ 空間	部門數 < 10 完全夠用，查詢 $O(1)$
鄰接表 $O(\text{edges})$	節省空間但查詢 $O(\text{degree})$
結論	小規模選矩陣（簡單），大規模才換鄰接表

從哪兩個資料源建構

```
function analyzeDeptNetwork(reports, departments, handoffs) {
  const depts = departments.filter(d => d.active).map(d => d.name)

  // ■■■■■■
  const matrix = {}
  depts.forEach(a => {
    matrix[a] = {}
    depts.forEach(b => matrix[a][b] = 0)
  })

  // (1) ■■■■■■ mention
  reports.forEach(r => {
    const text = `${r.cases} ${r.blockers} ${r.needHelp} ${r.nextWeek}`
    depts.forEach(target => {
      if (target === r.dept) return // ■■■■
      const count = (text.match(new RegExp(target, "g")) || []).length
      matrix[r.dept][target] += count
    })
  })

  // (2) ■■■■■■
  handoffs.forEach(h => matrix[h.from][h.to] += 1)

  return { matrix, depts }
}
```

#20 Force-directed Graph Layout 力導向佈局

● 圖論 · 視覺化

起源

1991 年 Fruchterman-Reingold 演算法。

D3.js 的 force simulation 同源。

物理基礎：庫倫定律（斥力）+ 虎克定律（彈簧吸力）。

參數怎麼定

<code>K_REP = 400</code>	節點間斥力常數，讓節點間維持 100-200 px 距離（適合 SVG 視窗大小）
<code>K_ATTR = 0.06</code>	邊吸引力，配合 K_REP 達到平衡
<code>damping = 0.85</code>	阻尼。0.5 太快收斂、0.95 振盪不收斂、0.85 是 sweet spot
<code>dt = 0.2</code>	時間步長，太大會抖動

主迴圈

```
for (let step = 0; step < iterations; step++) {  
  // (a)   
  for (each pair of nodes A, B) {  
    dist = distance(A, B)  
    force = K_REP / (dist * dist) //   
    A.velocity += repel_direction * force * dt  
  }  
  
  // (b)   
  edges.forEach(e => {  
    dist = distance(e.from, e.to)  
    force = K_ATTR * dist * e.weight //   
    both nodes' velocities += attract_direction * force * dt  
  })  
  
  // (c)   
  nodes.forEach(n => {  
    n.velocity *= damping  
    n.position += n.velocity  
  })  
}
```

⚠ 限制

1. 我們的 SVG 實作為簡化版（無 quadtree），部門數 > 20 會卡頓
2. 初始位置隨機，每次結果可能略不同
3. 大型網絡需要遷移到 D3 force simulation 套件

#21 Asymmetric Communication Detection

單向溝通偵測

● 圖論 · 組織病徵

起源

社會網絡分析 (SNA) 的「互惠性 (Reciprocity)」指標。

為什麼用嚴格 0

不是「A 提到 B 多 vs B 提到 A 少」這種比例不平衡（這可能正常），而是「A 提 B ≥ 5 次 AND B 完全沒提 A」這種完全黑洞 — 才是組織病徵。

閾值怎麼定

$A \rightarrow B \geq 5$ 5 次提及代表「經常性依賴」而非偶然。對應 z-score ≈ 1 （統計顯著性的非正式門檻）

$B \rightarrow A = 0$ 嚴格 0。不抓「不平衡」，只抓「完全沒回」— 前者可能正常，後者是組織病徵

實作

```
function detectAsymmetry(matrix, depts) {
  let count = 0, ratio = 0
  for (const a of depts) {
    for (const b of depts) {
      if (a === b) continue
      const ab = matrix[a]?.[b] || 0
      const ba = matrix[b]?.[a] || 0
      if (ab >= 5 && ba === 0) {
        count++
        ratio += ab
      }
    }
  }
  return { count, ratio }
}

// ■■■ ORI ■ CDC ■■■
CDC = 100 + asymCount × 18 + asymRatio × 0.5
```

⚠ 限制

1. 閾值 5 是 magic number，小團隊可能不適用
2. 未區分「該回但沒回」vs「本來就不該回」的對話
3. 依賴週報文字 mention 抽取，若某部門寫週報少會偵測不到

第六部 · 狀態判定類

#22 Decision Status Helpers 動態狀態判定

● 狀態判定 · v2.1 重大新增

起源

我們自創 (v2.1 修 bug 時加的)。

概念來自 DDD (領域驅動設計)：動態判定 > 靜態 status 欄位。

為什麼這樣設計

「狀態」應該是資料的函數，不是另一個資料欄位。

否則就會發生「dueDate 過期但 status 沒更新」的 bug — v2.1 就是因為這個發現 9 件 bug。

4 個 helper

```
isDecisionOverdueAt(d, asOf = NOW): boolean
    decidedAt ≤ asOf AND dueDate < asOf
    AND (!completedAt OR completedAt > asOf)

isDecisionInProgressAt(d, asOf): boolean
    decidedAt ≤ asOf AND NOT overdue AND NOT completed

isDecisionCompletedAt(d, asOf): boolean
    completedAt ≤ asOf

daysOverdue(d, asOf): number
    max(0, round((startOfDay(asOf) - startOfDay(dueDate)) / 86400000))
```

v2.2 加 startOfDay 對齊

原本 daysOverdue 計算受時分秒 / 時區殘留影響，可能算出「逾期 0.5 天」被四捨五入成 0 或 1，造成 UI 跨日跳動。v2.2 加 startOfDay 截斷到午夜零點，徹底避免邊界浮點誤差。

⚠ 限制

每次調用都要重算 (沒 cache) — 但因為都是純整數比較，O(1) 開銷可忽略。

#23 Risk / Load / Health Level Mapping

等級對應

● 狀態判定 · 視覺化橋樑

為什麼需要

作為「數據與用戶視覺」之間的語意橋樑 (Semantic Mapping)。將冰冷的統計數字（如 74.2 分、0.41 的 Gini）轉換為具有行動指導意義的燈號標籤。

三組 mapping

Blocker Level (critical / high / medium / normal)	依 P75 / P90 / P95 自動分類 — 見 #7 Empirical Percentile
---	--

Load Level (overload / high / normal / low / idle)	依絕對值 25/15/6/1 OR percentile 90/75 — 見 #14
--	--

Health Level (優異 / 良好 / 可關注 / 需注意 / 亟需介入)	依 overall $\geq 85 / 70 / 55 / 40 / < 40$ — 對齊台灣教育部評等
---	---

實作

```
function healthLevel(overall: number) {
  if (overall >= 85) return { label: "■■■", color: "text-emerald-600" }
  if (overall >= 70) return { label: "■■■", color: "text-blue-600" }
  if (overall >= 55) return { label: "■■■■", color: "text-amber-600" }
  if (overall >= 40) return { label: "■■■■", color: "text-orange-600" }
  return { label: "■■■■■", color: "text-red-600" }
}
```

第七部 · 預測模擬類

#24 What-if Scenario Simulation

情境模擬器

● 預測模擬 · v2.2 互動核心

起源

來自 Excel 的 What-if Analysis（70 年代發明）。

Decision Support Systems (DSS) 經典功能（Gorry & Scott Morton 1971）。

為什麼選 Shadow Data 模式

對原始資料 fork shadow data（不汙染主資料），套用 scenario 修改後，重跑 computeHealthSnapshot，與 baseline 雙圖層雷達比對。

純函數性：相同 scenario 永遠回傳相同模擬結果。

Scenario 結構

```
type Scenario = {  
  resolvedBlockerIds: Set<string>, // ████████  
  expeditedDecisionIds: Set<string>, // ██████████  
  signedHandoffIds: Set<string>, // ██████████  
  extraHeadcount: { [dept]: 0..5 } // ██████  
}
```

完整流程（含 v2.2 useDeferredValue）

```
// React 19 useDeferredValue ████████████████ UI  
const deferredScenario = useDeferredValue(scenario)  
const isPending = scenario !== deferredScenario // ██████████...█  
  
const shadowData = useMemo(() => ({  
  blockers: blockers.map(b =>  
    deferredScenario.resolvedBlockerIds.has(b.id)  
    ? { ...b, status: "resolved" } : b),  
  decisions: decisions.map(d =>  
    deferredScenario.expeditedDecisionIds.has(d.id)  
    ? { ...d, status: "████", completedAt: NOW } : d),  
  handoffs: handoffs.map(h =>  
    deferredScenario.signedHandoffIds.has(h.id)  
    ? { ...h, status: "████" } : h),  
  employees: [...employees, ...generateExtras(deferredScenario.extraHeadcount)],  
}), [deferredScenario])  
  
const projected = useMemo(() =>  
  computeHealthSnapshot(NOW, ...shadowData), [shadowData])  
  
const delta = projected.overall - baseline.overall
```

⚠ 限制

1. 假設「解掉卡點 = 立即生效」，現實中可能有延遲
2. 模擬「加員工」目前是新增 loadScore=0 的人，可能拉高 Gini（反效果）— 該改為 reassignment
3. 資料量 10× 時即使 useDeferredValue 也會卡，需 Web Worker

#25 Smart Suggestion 智能建議文案

● 預測模擬 · UX

起源

Decision Support System (DSS) 的「最後一哩路」— 把數字轉成可行動的文案。

五級閾值

$\Delta \geq +5$	顯著改善 — 強烈建議執行 — 5 是 100 分總分的 5%，已是明顯感受
$\Delta +2 \sim +5$	可考慮 — 2 = 雜訊上限以上
$\Delta -2 \sim +2$	影響不大 — 雜訊區間
$\Delta -5 \sim -2$	需評估 — 對稱負面區
$\Delta \leq -5$	⚠ 不建議 — 對稱「顯著」區

實作

```
function smartSuggestion(delta: number): string {  
  if (delta >= 5) return "■■■■ ■ ■■■■■■"  
  if (delta >= 2) return "■■■■■■■■■■"  
  if (delta >= -2) return "■■■■■■■■■■"  
  if (delta >= -5) return "■■■■■■■■"  
  return "■■■■ ■■ ■■■■■"  
}
```

第八部 · 工具機制類

#26 Optimistic UI Sync 樂觀同步

● 工具 · 資料層

起源

1990 年代 Optimistic Concurrency Control 起源於資料庫領域。
前端版本流行於 React 16+ 時代，目前 React Query / SWR 都採用。

為什麼用樂觀同步

讓使用者感覺「零延遲」－ UI 立即更新，背景才同步雲端。
悲觀同步：等 Firestore 回應才更新 UI → 300-500ms 延遲 → 體感不流暢。

實作

```
function useAppData() {
  const [reports, setReports] = useState(SEED_REPORTS)
  const [syncStatus, setSyncStatus] = useState<"idle"|"syncing"|"error">("idle")

  // user action → setState ■■■■ UI
  // useEffect ■■ → ■■■■ Firestore
  useEffect(() => {
    if (!dataLoaded) return
    setSyncStatus("syncing")
    saveDocumentCollection("reports", reports)
      .then(ok => setSyncStatus(ok ? "idle" : "error"))
  }, [reports])
}
```

#27 SEED Protection 種子資料保護

● 工具 · 容錯

起源

Defensive Programming 概念，可追溯到 1970 年代結構化程式設計。

為什麼需要

Firestore 集合可能：

- (1) 初始化時為空（首次登入）
- (2) 被誤刪（如新進工程師清空測試資料）
- (3) 包含舊版格式（「第 N 週」），與新版不相容

若不保護，前端會 `setDepartments([])` → 「`departments.filter(...)`」變空 → 整個 app 崩潰。

各 collection 的保護門檻

<code>reports / handoffs</code>	< 10 筆 → 用 SEED；含舊「第 N 週」格式也重置
<code>blockers / decisions / employees</code>	空時 fallback 到 SEED
<code>history / meetingHistory</code>	< 5 筆 → 用 SEED
<code>departments / users</code> (v2.1 新增)	空時 fallback — 避免 app 崩潰

實作

```
// reports / handoffs ■■ + ■■■■■■
const hasOldFormat = r.some(x => /\s*\d+\s*■/.test(x.week))
const finalReports = (hasOldFormat || r.length < 10) ? SEED_REPORTS : r

// ■■ collection ■■ fallback
const finalBlockers = b.length === 0 ? SEED_BLOCKERS : b
const finalHistory = hist.length < 5 ? SEED_HISTORY : hist

// v2.1 ■■
const finalDepts = deptRows.length === 0 ? SEED_DEPARTMENTS : deptRows
const finalUsers = userRows.length === 0 ? SEED_USERS : userRows
```

#28 NaN Guards / Clamp 數值安全保護

● 工具 · 全系統

起源

Floating-point safety 是程式語言設計基礎。

IEEE 754 標準 (1985) 定義 NaN / Infinity, 但實際使用時要主動防護。

三大工具

```
// (1) Clamp - [lo, hi]
const clamp = (v, lo = 0, hi = 100) => Math.max(lo, Math.min(hi, v))

// (2) Safe Date - NaN
function safeDate(s?: string): Date | null {
  if (!s || s === "") return null
  const d = new Date(s)
  return isNaN(+d) ? null : d
}

// (3) Safe Ratio - + clamp
function safeRatio(num, den, fallback = 0): number {
  if (den <= 0) return fallback
  return Math.min(1.0, num / den) // submission rate max 1
}
```

應用情境

所有評分函式末段	clamp(score, 0, 100) 確保不溢出
日期 parse	「即時生效」、「null」、空字串都要擋
除法	空 array 算 mean 時分母為 0 — safeRatio 擋住
submission rate	管理層交週報時可能 > 1, 要 clamp

#29 Linear Regression Baseline Drift Slope

線性回歸基準漂移

● v2.2 新工具 · Cohort 核心

起源

1805 年 Adrien-Marie Legendre 提出最小平方法 (Method of Least Squares)。

1809 年 Carl Friedrich Gauss 給出正式統計證明。

現在是統計學最基本工具，所有 spreadsheet 都內建。

為什麼用線性回歸

教授必問：「為什麼不用前後兩週直接相減除以天數算出變化就好？」

答：因為單一週的數據極易包含隨機雜訊（如某週剛好放連續假期導致指標失真）。線性回歸能有效利用過去 12 週的所有資料點，互相抵消短期隨機雜訊，抓出最穩健、最不失真的長期方向性趨勢。

公式（最小平方法）

■ N ■ (x■, y■)■

■ m = $\frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$

■ $\bar{x} = \text{mean}(x)$, $\bar{y} = \text{mean}(y)$

→ ■■■■■ m ■■■■■■ c ■■■

完整實作

```
function linearRegressionSlope(points: {x, y}[]): number {
  const n = points.length
  if (n < 2) return 0

  // 1. ■■■
  const meanX = stats.mean(points.map(p => p.x))
  const meanY = stats.mean(points.map(p => p.y))

  // 2. ■■■■■
  let numerator = 0, denominator = 0
  for (const p of points) {
    const devX = p.x - meanX
    const devY = p.y - meanY
    numerator += devX * devY
    denominator += devX * devX
  }

  // 3. ■■■■■■■■ X ■■■
  return denominator === 0 ? 0 : numerator / denominator
}

// ■ Decision Impact ■■■
function computeBaselineDriftSlope(data, cache) {
  if (cache.ratePerDay !== undefined) return cache.ratePerDay

  // ■■ 12 ■■■
  const samples = []
  for (let i = 0; i < 12; i++) {
    const asOf = new Date(NOW)
    asOf.setDate(asOf.getDate() - (11 - i) * 7)
    const snap = computeHealthSnapshot(asOf, ...data)
    samples.push({
```

```
        x: (+asOf - +NOW) / 86400000, // ■ NOW ■■■■■■  
        y: snap.overall  
    })  
}  
  
cache.ratePerDay = linearRegressionSlope(samples)  
return cache.ratePerDay  
}
```

⚠ 限制

1. 假設線性趨勢 — 若組織有 S 型成長或週期性，會失真
2. 12 週樣本對「跨季度」事件可能不夠
3. Slope 對首尾極端點敏感 (leverage effect) — 業界做法可換 Theil-Sen estimator

結語 · 演算法選擇的設計哲學

本系統 29 個演算法共同實現一個目標：
「用恰當的演算法解決恰當規模的問題」。

4 條核心原則

1. 學術可考	BM25F、Gini、Linear Regression 等都有 100+ 年歷史，論文可查
2. 經驗有據	Time decay 半衰期 = Andy Grove 注意力週期、Gini 0.35 = Lambert 2001
3. 可解釋性優先	每個分數都能逐層拆解，不用黑盒模型
4. 跨頁面一致	同一個指標在不同頁面用同一個分析器，避免「兩邊數字對不上」

為什麼不用 LLM / Embedding

- (1) 資料量小：53 筆歷史案、200 筆週報。BM25F 比 Embedding 更可靠且零成本。
- (2) 可解釋性：管理層需要知道「為什麼推薦這筆」，BM25F 可逐項列出命中詞貢獻。
- (3) 機密敏感：投資資料不能送 cloud API。
- (4) 計算成本：所有演算法都是 $O(N)$ 或 $O(N \log N)$ 。
- (5) 確定性：相同輸入永遠回傳相同結果，LLM 有 stochasticity。

「資料越少，演算法的選擇越重要。」
「跨頁面的一致性，比單頁的炫技更重要。」
「能解釋的演算法，比準確 1% 的黑盒更值錢。」

—— 串連系統 v2.2 設計哲學